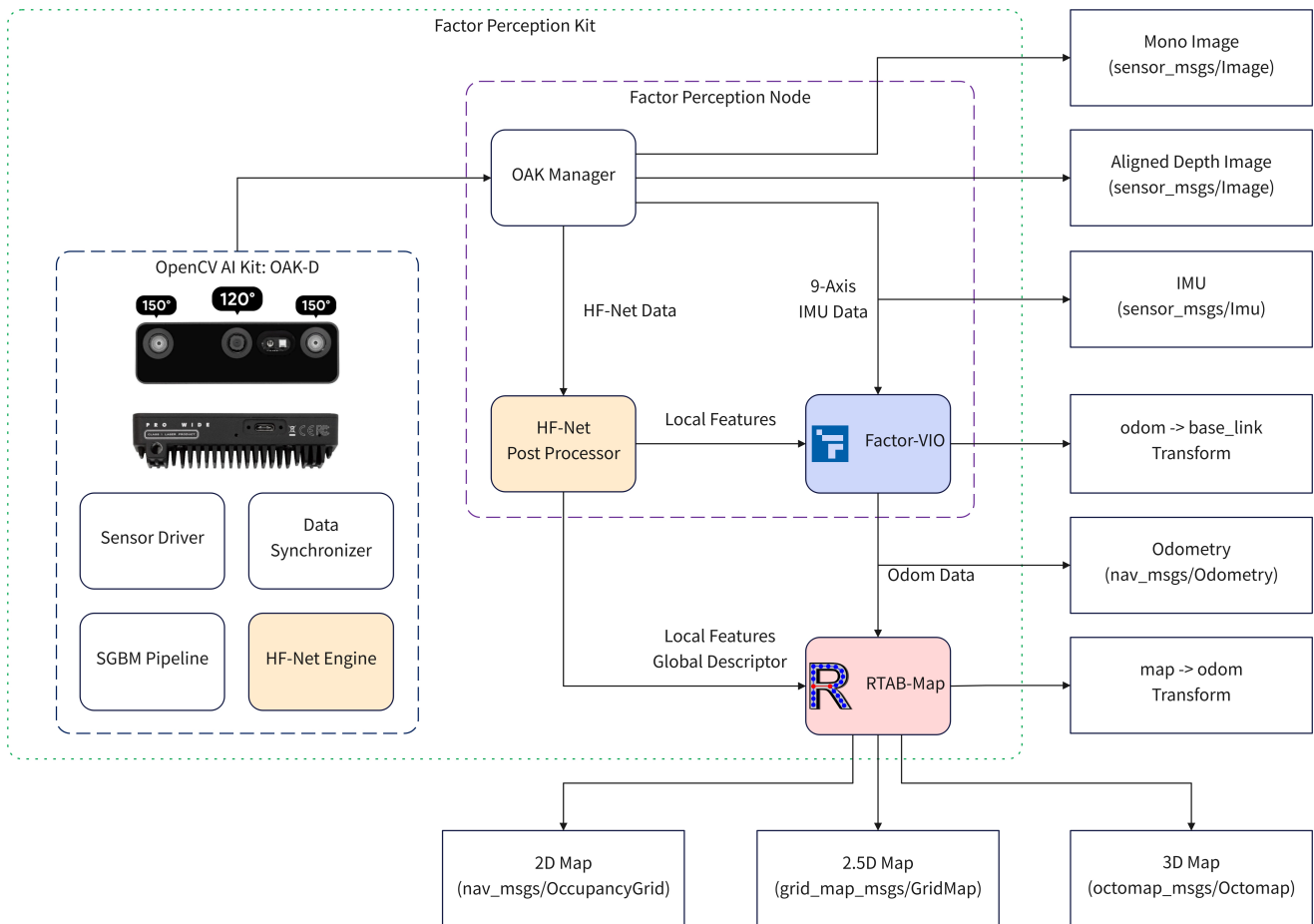


Factor Perception User Guide

1 Introduction

Welcome to factor perception SDK. This is a ROS package for robot spatial perception with AI visual SLAM as the core. Factor Perception provides a SOTA 3D SLAM solution while being very easy to use. You can experience plug-and-play SLAM with supported hardware, or integrate it into various robots with just a few simple configuration steps. Factor Perception includes the closed-source Factor-VIO front-end and uses the open-source RTAB-Map as the back-end. Its core developer is also a contributor to the RTAB-Map library. Factor Perception provides centimeter-level localization accuracy and outputs at up to 200 Hz. This is a true AI-assisted SLAM system. The front-end and back-end do not use raw images directly, but use data processed by the neural network. Therefore, the system is extremely robust, has been tested in a variety of extreme scenarios, and has almost no range and scenario limitations. In addition, Factor Perception also integrates advanced technologies such as hybrid SLAM, lifelong SLAM, memory management, incremental smoothing and mapping (iSAM), which can cope with various long-term and short-term environmental changes, without manual position initialization, and can realize multi-robot SLAM and automatic map merging. The core functions of SLAM take up very few hardware resources and only require a Raspberry Pi-level host computer to run in real time. Therefore, our entire solution no longer provides additional computing power platforms. Although AI-assisted, no GPU is needed, and the neural network inference runs in the camera. Factor Perception strictly follows REPs and can seamlessly replace other SLAM or localization frameworks such as GMapping, Cartographer, AMCL. It is closely integrated with the ROS navigation stack and can provide 2D map (OccupancyGrid), 2.5D map (GridMap), and 3D map (OctoMap) to meet the needs of different robot applications. With the high openness of the RTAB-Map back-end, users can also easily integrate other data sources such as lidar, GPS, QR codes, and realize other functions such as 3D reconstruction. The following figure shows the overall framework of Factor Perception.



Version 1.5.1

2 Software and Hardware Requirements

Factor Perception currently supports ROS1 Noetic, ROS2 Humble and Jazzy platforms, and the hardware uses OAK-D series cameras. We recommend the wide FOV models (marked in bold below), and for use at night or in low-light environments, the Pro models with infrared capabilities are available (marked in *italics* below).

Tested models: OAK-D S2, **OAK-D W**, *OAK-D Pro*, **OAK-D Pro W**, OAK-D S2 PoE, **OAK-D W PoE**, *OAK-D Pro PoE*, **OAK-D Pro W PoE**, OAK-D LR, OAK-D SR

You can get more information about the hardware from Luxonis documentation.

📄 <https://docs.luxonis.com/hardware/>

3 Installation and Configuration

3.1 ROS Package Installation

We assume that you have completed the ROS environment installation and understand the functions and usage of ROS. If you are not familiar with ROS, you can refer to the ROS Wiki.

📄 <https://wiki.ros.org>

Factor Perception SDK is provided as a standalone ROS package. We have packaged it as a deb package. You can use dpkg, apt or other tools to install:

```
$ cd /path/to/deb/file/
$ sudo dpkg -i ros-<distro>-factor-perception_<version>_<architecture>.deb
```

Other required dependencies are checked automatically during installation. Factor Perception's dependencies are all from Ubuntu repository or ROS repository, and no third-party dependencies are used. If you are prompted with missing dependencies after installing with `dpkg`, you can use `apt-get` to automatically fix the required dependencies:

```
$ sudo apt-get -f install
```

3.2 udev Rule Configuration

If the OAK camera is connected via USB, the user needs to manually add a udev rule to grant access permissions to the device:

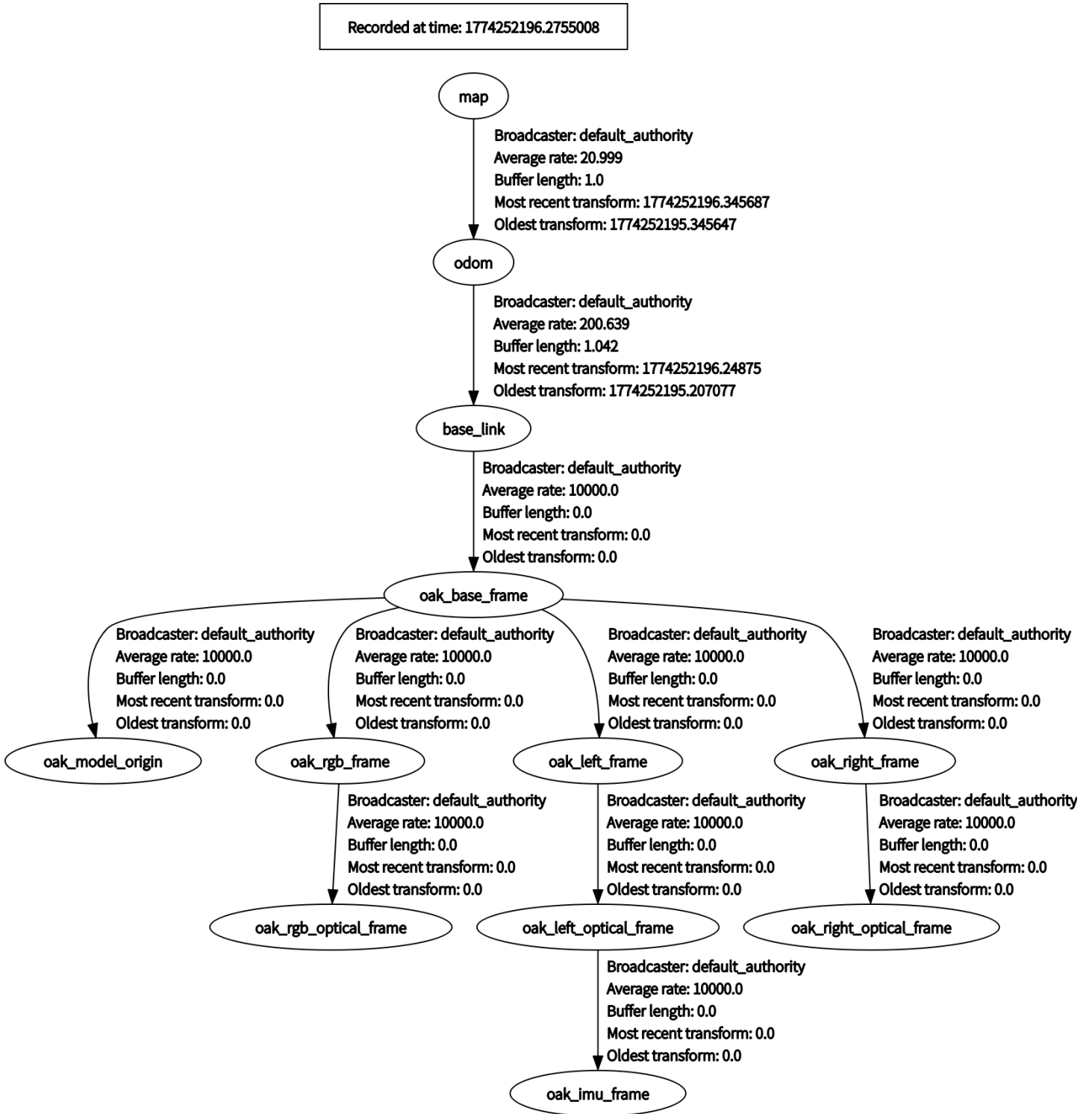
```
$ echo 'SUBSYSTEM=="usb", ATTRS{idVendor}=="03e7", MODE="0666"' | sudo tee /etc/udev/rules.d/80-movidius.rules
$ sudo udevadm control --reload-rules && sudo udevadm trigger
```

3.3 Functionality Authorization

Enabling the full functionality of Factor Perception requires a key bound to the OAK camera. You can save the key in a launch file or other configuration file and provide it as a parameter each time you start the system. If the correct key is not provided when the system is started, the camera raw data will still be output to facilitate testing and verification.

3.4 Robot Model Configuration

Factor Perception provides Xacro and STL models of OAK cameras for generating URDF. Users should generally integrate this into their own robot models. When using the default TF prefix, the TF Tree is as shown below. The base frame of the OAK camera is `oak_base_frame`, and two frames are defined for each individual camera, namely the camera frame and the optical frame. The origins of both frames are located at the camera optical center, and the orientation follows [REP-0103](#). That is, the camera frame has the X-axis facing forward, the Y-axis faces left, and the Z-axis faces up, the optical frame has the X-axis facing right, the Y-axis faces down, and the Z-axis faces forward. The camera's intrinsics and extrinsics are parsed from the device's factory calibration values. Users only need to configure the transform from the robot frame (such as `base_link`) to the camera base frame. Factor Perception will directly provide the transform from the odometry frame (`odom`) to the robot frame (`base_link`), and the map frame (`map`) to the odometry frame (`odom`). Therefore, users can directly obtain the localization of the robot frame without making any other



3.5 Additional Middleware Configuration Required on ROS2 Platform

Communication in ROS2 is based on the Data Distribution Service (DDS) by default. Based on the DDS specification, there are multiple middleware implementations supported by ROS2. The default middleware used in ROS2 Humble and ROS2 Jazzy is eProsima's Fast DDS. However, the current version of Fast DDS is not reliable when transferring large amounts of data. For example, it is well known that [Navigation 2 is not safe to use with Fast DDS](https://docs.ros.org/en/jazzy/Installation/RMW-Implementations/DDS-Implementations/Working-with-Eclipse-CycloneDDS.html). Our system also needs to transfer a large amount of data, so we will encounter similar problems. The current solution is to switch to other DDS middleware and perform network tuning, or use a non-DDS middleware, to ensure stable communication. The most recommended is to use Cyclone DDS or Zenoh. The installation and switching steps are very simple, please refer to the ROS2 documentation.

❏ <https://docs.ros.org/en/jazzy/Installation/RMW-Implementations/DDS-Implementations/Working-with-Eclipse-CycloneDDS.html>

❏ <https://docs.ros.org/en/jazzy/Installation/RMW-Implementations/Non-DDS-Implementations/Working-with-Zenoh.html>

If you switch to other DDS middleware, you still need to perform network tuning to reduce communication overhead. Relevant instructions and suggestions are also provided in the ROS2 documentation.

❏ <https://docs.ros.org/en/jazzy/How-To-Guides/DDS-tuning.html>

4 Functional Description

4.1 Launch the Sample Launch File

The sample launch file provides a minimal set of running configurations to launch, test and verify core functionality with a single line of command. For ROS1:

```
$ roslaunch factor_perception factor_perception.launch key:=<your factor_perception key>
```

For ROS2:

```
$ ros2 launch factor_perception factor_perception_launch.py key:=<your factor_perception key>
```

You can start the system while the camera is still or moving. Factor-VIO will perform static or dynamic initialization automatically. For static initialization, the camera should remain still for a few seconds and avoid facing areas that lack texture, such as a white wall. For dynamic initialization, the camera should move continuously and be fully excited in all 6 degrees of freedom.

You can also include the sample launch file in your own launch file, or use it as a template. In addition to the parameters introduced in 4.2.2 that are passed directly to the Factor Perception Node, the sample launch file also declares additional parameters. You can modify **camera_model** to use the corresponding camera STL model in the URDF. You can configure the mounting position of the OAK camera, which is the transform from *base_link* to *oak_base_frame*. In this way, the system can directly provide the pose and velocity of *base_link*. Additionally, you can choose whether to run the system in SLAM mode or Localization mode, and whether to use [rtabmap_viz](#) for visualization. **rtabmap_viz** can also be launched separately on the local computer or on other computers in the ROS network.

For ROS1:

```
$ roslaunch factor_perception rtabmap_viz.launch
```

For ROS2:

```
$ ros2 launch factor_perception rtabmap_viz_launch.py
```

If **rtabmap_viz** is not usable (usually due to driver issues on some platforms), you can also use **rviz** or **rviz2** for visualization. You need to install [rtabmap_rviz_plugins](#) additionally to visualize custom data of RTAB-Map. To visualize OctoMap, you also need to install [octomap_rviz_plugins](#). They are both available from the ROS repository. The rviz reference config is stored in:

```
/opt/ros/<distro>/share/factor_perception/config/factor_perception.rviz
```

4.2 Factor-VIO Front-end

4.2.1 Published Topics

rgb/image_rect/compressed ([sensor_msgs/CompressedImage](#))

*Rectified color image from the center camera with output rate of 20 Hz. This topic is not available when using OAK-D LR or OAK-D SR. Directly publish JPEG images encoded in the OAK camera. Other nodes can use [image_transport](#) to subscribe to **rgb/image_rect** and choose **compressed** transport.*

rgb/camera_info ([sensor_msgs/CameraInfo](#))

Intrinsics of the center camera. This topic is not available when using OAK-D LR or OAK-D SR.

left/image_rect/compressed ([sensor_msgs/CompressedImage](#))

*Rectified image from the left camera with output rate of 20 Hz. Color image is published when using OAK-D LR or OAK-D SR, monochrome image is published when using other models. Directly publish JPEG images encoded in the OAK camera. Other nodes can use [image_transport](#) to subscribe to **left/image_rect** and choose **compressed** transport.*

left/camera_info ([sensor_msgs/CameraInfo](#))

Intrinsics of the left camera.

depth/image_rect ([sensor_msgs/Image](#))

Rectified depth image with output rate of 20 Hz, aligned to the left camera.

depth/camera_info ([sensor_msgs/CameraInfo](#))

Intrinsics of the depth camera.

rgbd_image ([rtabmap_msgs/RGBDImage](#))

Contains synchronized image from the left camera, depth image, camera intrinsics, local features and descriptors, global descriptor.

imu ([sensor_msgs/Imu](#))

9-axis IMU data with output rate of 200 Hz. Contains 3-axis acceleration, 3-axis angular velocity, and 9-axis fused orientation.

odom ([nav_msgs/Odometry](#))

Visual inertial odometry data with output rate of 200 Hz, synchronized with IMU data.

4.2.2 Parameters

-mxid_or_name (string, default: "")

MXID, or IP address, or USB port name of the OAK camera. The default is empty, and the first OAK camera automatically discovered will be used. Device can be specified when connecting multiple OAK cameras.

-key (string, default: "")

The key to enable full functionality.

-oak_tf_prefix (string, default: "oak")

OAK camera TF prefix.

-base_frame_id (string, default: "base_link")

Robot frame.

-odom_frame_id (string, default: "odom")

Odometry frame.

-camera_cpu (int, default: -1)

CPU ID to which the camera thread is bound. A negative value indicates that it is not bound to a specific CPU core.

-imu_cpu (int, default: -1)

CPU ID to which the IMU thread is bound. A negative value indicates that it is not bound to a specific CPU core.

-publish_tf (bool, default: "true")

Publish TF from the odometry frame to the robot frame. By default, visual inertial odometry data is published through both ROS topics and TF. If the user intends to fuse other sensor or odometry data, TF publishing can be disabled and the fusion node can publish it instead. It should be noted that multi-sensor fusion that is not well modeled may actually reduce odometry accuracy.

-depth_filter (bool, default: "false")

Use a specially designed high-confidence filtering algorithm to obtain depth data with extremely low false alarm rates. This can help build cleaner maps and provide higher quality data for navigation and obstacle avoidance, but will slightly increase CPU usage.

-ir_intensity (double, default: 0.0)

Infrared floodlight intensity (>0.0 and <1.0), only valid for Pro models with infrared capabilities. Can improve SLAM performance in low-light environments.

-min_feat_depth (double, default: 0.0)

Minimum depth of features. Features below this distance will be filtered out, since they are generally fixed features caused by occlusion of the carrier.

-blob_path (string, default: "")

*The neural network model used for SLAM, will be loaded into the camera when the system starts. The default is empty, using the traditional feature detection and tracking algorithm. Or use **HF-Net.blob** in the blobs directory of the ROS package to enable deep learning-based feature detection and visual place recognition. Refer to the sample launch file configuration.*

4.2.3 Required tf Transforms

base_link → oak_base_frame

The mounting position of the OAK camera on the robot. Usually static, but can also be dynamic. For example, a camera mounted on a robotic arm can be used for SLAM, as long as the kinematic information of the robotic arm is provided in real time through TF.

4.2.4 Provided tf Transforms

odom → base_link

The pose estimation of the robot in the odometry frame.

4.3 RTAB-Map Back-end

Factor-VIO front-end uses the custom data of RTAB-Map to transmit image data and processed local and global features to the back-end. This deep integration avoids some duplicate computations in the RTAB-Map back-end and enables some of its advanced features. Therefore, we need to adjust some default configurations of RTAB-Map. The RTAB-Map reference configuration provided by Factor Perception is stored in:

```
/opt/ros/<distro>/share/factor_perception/config/rtabmap.ini
```

You can change the **config_path** parameter to use another configuration file. We recommend changing parameters with **Grid** or **GridGlobal** prefix according to application needs. Other parameters should use the settings in the reference configuration and users are not recommended to modify them. You can run the following command alone to see all available parameters and their descriptions for the RTAB-Map back-end.

For ROS1:

```
$ rosrunc rtabmap_slam rtabmap --params
```

For ROS2:

```
$ ros2 run rtabmap_slam rtabmap --params
```

The map data of RTAB-Map is stored in a database with its own internal representation. It can be converted into other map representations as needed when used. The storage location of the database can be set by the **database_path** parameter, defaults to "~/ros/rtabmap.db". We change it to "~/rtabmap.db" in the sample launch file for convenience. RTAB-Map supports multi-session mapping. When the same map database is used all the time, a new session is created each time it is started. RTAB-Map attempts to match and merge maps between sessions to achieve incremental mapping and lifelong SLAM. You need to manage the maps yourself based on your application needs. We don't recommend storing maps of different regions in the same database unless they can be merged later. You can use the [rtabmap-databaseViewer](#) tool to browse the map, perform offline optimization, export and perform 3D reconstructions.

The RTAB-Map back-end is completely open source. For more information, see the [GitHub Wiki](#) and the [ROS Wiki](#).

4.4 Sensor Calibration

4.4.1 IMU Self-Calibration

When performing dynamic initialization, Factor-VIO estimates the bias of the accelerometer and gyroscope. When performing static initialization, only the gyroscope bias can be accurately estimated, while the accelerometer bias can only be roughly estimated. This is because to estimate the accelerometer bias we need to remove the gravity component, and to accurately estimate the direction of gravity we need to know the bias first. Therefore, to perform a good static initialization, we need to eliminate most of the accelerometer bias in advance, mainly the constant bias. For the BNO085/086 IMU used in the OAK camera, the six-position method can be used for self-calibration. Some models and batches of OAK cameras do not undergo IMU self-calibration at the factory, which results in a large bias in the acceleration output. If you find that the system accuracy and robustness are lower than expected during use, you can reperform IMU self-calibration. Please follow the steps below:

- Launch the Sample Launch File
- Place the camera in six orientations on a horizontal plane (e.g., X-axis positive, X-axis negative, Y-axis positive, Y-axis negative, Z-axis positive, Z-axis negative), and remain stationary for at least 1 second in each orientation.
- At this point, the IMU firmware has completed self-calibration, but the calibration data has not yet been saved.
- Use `Ctrl+C` in the terminal to terminate the system, do not unplug the camera (always keep the camera powered on).
- Relaunch the sample Launch file, the calibration data will be saved into flash.

For more information on IMU self-calibration, please refer to the CEVA documentation.

📄 <https://www.mouser.com/pdfDocs/BNO080-BNO085-Sensor-Calibration-Procedure.pdf>

4.4.2 Dynamic Extrinsic Calibration

OAK cameras do not calibrate the extrinsics between the camera and the IMU at the factory. When starting the Factor-VIO frontend, you will receive the following warning: "IMU calibration data is not available on device yet." At this point, the system will use the extrinsics design values. This generally does not have a significant impact on system performance. However, if you are looking for higher accuracy and better consistency between camera and IMU data, you can use the sensor calibration node for dynamic extrinsics calibration. The sensor calibration node can be started using the sensor calibration launch file. The default mode is extrinsics calibration with rviz visualization enabled.

For ROS1:

```
$ roslaunch factor_perception sensor_calibration.launch
```

For ROS2:

```
$ ros2 launch factor_perception sensor_calibration_launch.py
```

To complete the extrinsics calibration, you need to shake the camera back and forth after the system initialization is complete. In rviz, the frames of the left and right cameras and the IMU will be displayed. You should see that the poses of these frames initially jitter, then converge to a stable state. The terminal will output the sigma value of extrinsics, and you should see it gradually decrease. When this value is less than 0.001, the calibration will automatically terminate, and the result will be flashed into camera's EEPROM. To achieve the best calibration results, the following points should be noted:

- Calibration can be performed in any environment, but requires the environment to have rich, non-repeating textures.
- Sufficient lighting should be provided in the environment to reduce image noise and motion blur.
- The camera should not only scan close-up features, but should also scan distant features.
- When shaking the camera, do not shake it too fast or too slow. It should include various translation and rotation movements to fully stimulate six degrees of freedom of the IMU.
- If you are not satisfied with the calibration results, you can repeat the calibration process. The system will use the results of the previous round as the initial value for the new round.

4.4.3 Reset to Factory Calibration

The sensor calibration node only modifies the user calibration data in the EEPROM. You can restore it to factory calibration at any time. Simply set **calibration_mode** to **reset** when launching the sensor calibration launch file.

For ROS1:

```
$ roslaunch factor_perception sensor_calibration.launch calibration_mode:=reset rviz:=false
```

For ROS2:

```
$ ros2 launch factor_perception sensor_calibration_launch.py calibration_mode:=reset rviz:=false
```